

PostgreSQL Roles and Privileges

Table of Contents

1. [Learning Objectives](#)
 2. [Role System Architecture](#)
 3. [Creating and Managing Roles](#)
 4. [Role Hierarchy and Inheritance](#)
 5. [Database Privileges](#)
 6. [Schema Privileges](#)
 7. [Table and Column Privileges](#)
 8. [Function and Sequence Privileges](#)
 9. [Default Privileges](#)
 10. [Superuser and Special Roles](#)
 11. [Privilege Inspection Queries](#)
 12. [Row-Level Security Preview](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Interview Questions](#)
 16. [Exercises and Solutions](#)
-

Learning Objectives

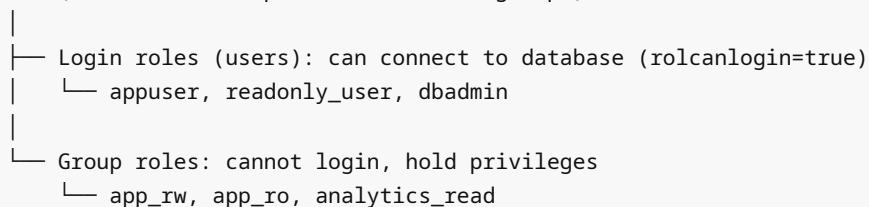
By the end of this module, you will be able to:

- Create roles with appropriate attributes and privileges
 - Design a role hierarchy using group roles
 - Grant and revoke privileges at all granularity levels
 - Use default privileges for consistent access control
 - Inspect and audit existing privileges
 - Implement principle of least privilege in PostgreSQL
-

Role System Architecture

PostgreSQL RBAC Model:

ROLES (unified – no separate "user" vs "group")



PRIVILEGE HIERARCHY:

Superuser → bypasses all privilege checks
|
pg_read_all_data → SELECT on all tables
pg_write_all_data → INSERT/UPDATE/DELETE on all tables

```
|
Database-level → CONNECT, CREATE
|
Schema-level → USAGE, CREATE
|
Table-level → SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER
|
Column-level → SELECT(col), UPDATE(col), REFERENCES(col)
|
Row-level → Row Level Security (RLS) policies
```

Creating and Managing Roles

CREATE ROLE Attributes

```
-- Full syntax
CREATE ROLE rolename
[ WITH ]
[ SUPERUSER | NOSUPERUSER ]           -- Bypass all permission checks
[ CREATEDB | NOCREATEDB ]             -- Can create databases
[ CREATEROLE | NOCREATEROLE ]         -- Can create/modify roles
[ INHERIT | NOINHERIT ]               -- Inherit privileges from parent roles
[ LOGIN | NOLOGIN ]                  -- Can connect to database
[ REPLICATION | NOREPLICATION ]      -- Can initiate replication
[ BYPASSRLS | NOBYPASSRLS ]          -- Bypass row-level security
[ CONNECTION LIMIT connlimit ]       -- Max simultaneous connections (-1 =
unlimited)
[ ENCRYPTED PASSWORD 'password' ]      -- Set password
[ VALID UNTIL 'timestamp' ]          -- Password expiration
[ IN ROLE rolename [, ...] ]         -- Immediately add to parent roles
[ ROLE rolename [, ...] ]            -- Immediately make parent of these roles
[ ADMIN rolename [, ...] ]           -- Admin option for these memberships
;
```

Common Role Creation Patterns

```
--
-- APPLICATION USER (limited login role)
--
CREATE ROLE appuser WITH
LOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE
NOREPLICATION
NOBYPASSRLS
CONNECTION LIMIT 50
PASSWORD 'StrongPassword123!'
VALID UNTIL '2025-12-31';
```

```

--
-- GROUP ROLE (no login, holds privileges)
--
CREATE ROLE app_readwrite;    -- Group role: no LOGIN
CREATE ROLE app_readonly;    -- Read-only group

--
-- SERVICE ACCOUNT (for microservices)
--
CREATE ROLE svc_orders WITH
    LOGIN
    NOCREATEDB
    NOCREATEROLE
    CONNECTION LIMIT 20
    PASSWORD 'ServiceAccountPass!';

--
-- READ-ONLY USER
--
CREATE ROLE readonly_user WITH
    LOGIN
    NOSUPERUSER
    CONNECTION LIMIT 10
    PASSWORD 'ReadOnlyPass!';

--
-- DBA ROLE (no superuser, but can manage databases)
--
CREATE ROLE dbadmin WITH
    LOGIN
    NOSUPERUSER
    CREATEDB
    CREATEROLE
    PASSWORD 'DBAPass!';

-- Managing roles:
ALTER ROLE appuser WITH PASSWORD 'NewPassword!';
ALTER ROLE appuser WITH CONNECTION LIMIT 100;
ALTER ROLE appuser VALID UNTIL 'infinity';    -- Remove expiration
ALTER ROLE appuser RENAME TO new_appuser;
DROP ROLE appuser;

```

Role Hierarchy and Inheritance

```

-- Create group roles (no login)
CREATE ROLE app_read;
CREATE ROLE app_write;
CREATE ROLE app_admin;

```

```

-- Create individual users
CREATE ROLE user_alice WITH LOGIN PASSWORD 'AlicePass!';
CREATE ROLE user_bob WITH LOGIN PASSWORD 'BobPass!';
CREATE ROLE user_charlie WITH LOGIN PASSWORD 'CharliePass!';

-- Assign users to groups
GRANT app_read TO user_alice;      -- Alice can read
GRANT app_write TO user_bob;      -- Bob can write (and read if app_read is also
granted)
GRANT app_read, app_write TO user_bob;
GRANT app_admin TO user_charlie;  -- Charlie is admin

-- Role hierarchy diagram:
-- app_admin —inherits—> app_write —inherits—> app_read
-- app_read  → SELECT on tables
-- app_write → INSERT, UPDATE, DELETE (inherits SELECT from app_read)
-- app_admin → TRUNCATE, REFERENCES (inherits write and read)

GRANT app_read TO app_write;      -- write inherits read privileges
GRANT app_write TO app_admin;     -- admin inherits write privileges

```

Inheritance Behavior

```

-- INHERIT (default): role automatically gets privileges of parent roles
CREATE ROLE reader WITH INHERIT;  -- gets SELECT when granted app_read

-- NOINHERIT: must SET ROLE to activate parent privileges
CREATE ROLE auditor WITH NOINHERIT LOGIN PASSWORD 'AuditorPass!';
GRANT app_read TO auditor;

-- Without inherit, auditor must explicitly switch roles:
-- SET ROLE app_read;    -- Now has app_read privileges
-- RESET ROLE;          -- Back to original

-- Check current role and effective privileges
SELECT current_user, session_user;

```

WITH ADMIN OPTION

```

-- Allow a role to grant membership in the group to others
GRANT app_admin TO user_charlie WITH ADMIN OPTION;
-- Now user_charlie can run: GRANT app_admin TO other_user;

-- Revoke admin option without revoking membership
REVOKE ADMIN OPTION FOR app_admin FROM user_charlie;

```

Database Privileges

```

-- CONNECT: Allow role to connect to a database
GRANT CONNECT ON DATABASE mydb TO appuser;

-- CREATE: Allow role to create schemas in a database
GRANT CREATE ON DATABASE mydb TO dbadmin;

-- TEMPORARY: Allow role to create temp tables
GRANT TEMPORARY ON DATABASE mydb TO appuser;

-- Grant all database privileges
GRANT ALL ON DATABASE mydb TO appuser;

-- Revoke public connect (prevent unrestricted database access)
REVOKE CONNECT ON DATABASE mydb FROM PUBLIC;

-- Only specific roles can connect
GRANT CONNECT ON DATABASE mydb TO app_read, app_write;

-- Check database privileges
SELECT datname, datacl FROM pg_database WHERE datname = 'mydb';
-- datacl: array of privilege descriptors
-- Format: rolname=privs/grantor
-- =Tc/postgres means PUBLIC has Tc from postgres

```

Schema Privileges

```

-- USAGE: Allow role to access objects within a schema
GRANT USAGE ON SCHEMA public TO appuser;
GRANT USAGE ON SCHEMA analytics TO analytics_read;

-- CREATE: Allow role to create objects in a schema
GRANT CREATE ON SCHEMA public TO dbadmin;
GRANT CREATE ON SCHEMA app_schema TO app_write;

-- Revoke public schema creation (important security hardening!)
REVOKE CREATE ON SCHEMA public FROM PUBLIC;

-- Grant usage and create together
GRANT USAGE, CREATE ON SCHEMA app_schema TO app_write;

-- Typical application setup:
-- App user needs USAGE (to see objects) + object-level grants
GRANT USAGE ON SCHEMA public TO appuser;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO appuser;

```

Table and Column Privileges

Table Privileges

```
-- Available table privileges:
-- SELECT      - Read rows
-- INSERT      - Add rows
-- UPDATE      - Modify rows
-- DELETE      - Remove rows
-- TRUNCATE    - Remove all rows (fast DELETE)
-- REFERENCES  - Create foreign key references
-- TRIGGER     - Create triggers

-- Grant to specific table
GRANT SELECT ON TABLE orders TO app_read;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE orders TO app_write;

-- Grant to all tables in schema
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_read;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_write;

-- Grant to app_admin group
GRANT ALL ON ALL TABLES IN SCHEMA public TO app_admin;

-- Revoke
REVOKE DELETE ON TABLE orders FROM app_write;
REVOKE ALL ON TABLE sensitive_data FROM PUBLIC;

-- VIEW privileges (same as tables)
GRANT SELECT ON VIEW order_summary TO reporting_role;

-- Check table privileges
SELECT grantee, table_name, privilege_type
FROM information_schema.table_privileges
WHERE table_schema = 'public'
ORDER BY grantee, table_name;
```

Column Privileges

```
-- Restrict access to specific columns
GRANT SELECT (id, name, email, created_at) ON TABLE users TO support_staff;
-- NOT GRANTED: password_hash, ssn, phone, address

GRANT UPDATE (email, phone) ON TABLE users TO profile_manager;
-- Only these columns can be updated

-- Column-level references (for foreign keys)
GRANT REFERENCES (id) ON TABLE users TO orders_service;

-- Verify column privileges
SELECT grantee, table_name, column_name, privilege_type
FROM information_schema.column_privileges
```

```
WHERE table_schema = 'public'
ORDER BY table_name, column_name;
```

Function and Sequence Privileges

```
-- Function privileges
GRANT EXECUTE ON FUNCTION calculate_tax(numeric) TO app_read;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO app_write;
REVOKE EXECUTE ON FUNCTION admin_reset_passwords() FROM PUBLIC;

-- SECURITY DEFINER functions (run as function owner, not caller)
CREATE OR REPLACE FUNCTION sensitive_report()
RETURNS TABLE(id int, total numeric)
SECURITY DEFINER    -- Runs as the function creator's privileges
SET search_path = public
AS $$
    SELECT id, total FROM sensitive_orders; -- Caller doesn't need SELECT on
sensitive_orders
$$ LANGUAGE sql;

-- Grant execute to limited users
GRANT EXECUTE ON FUNCTION sensitive_report() TO reporting_user;

-- Sequence privileges
GRANT USAGE, SELECT ON SEQUENCE orders_id_seq TO app_write;
GRANT ALL ON ALL SEQUENCES IN SCHEMA public TO app_write;
```

Default Privileges

Default privileges automatically grant permissions to future objects created in a schema.

```
-- Set defaults for objects created by a specific role
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA public
    GRANT SELECT ON TABLES TO app_read;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_write;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO app_write;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA public
    GRANT EXECUTE ON FUNCTIONS TO app_write;

-- View current default privileges
SELECT * FROM pg_default_acl;

-- More specific example: new tables in analytics schema
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE analytics_admin IN SCHEMA analytics
GRANT SELECT ON TABLES TO analytics_read;
```

Complete Setup Script: New Application

```
-- Complete privilege setup for a new application

-- 1. Create group roles
CREATE ROLE myapp_read;
CREATE ROLE myapp_write;
CREATE ROLE myapp_admin;

-- 2. Group hierarchy
GRANT myapp_read TO myapp_write;
GRANT myapp_write TO myapp_admin;

-- 3. Create login roles
CREATE ROLE myapp_api WITH LOGIN PASSWORD 'ApiPass123!' CONNECTION LIMIT 50;
CREATE ROLE myapp_worker WITH LOGIN PASSWORD 'WorkerPass123!' CONNECTION LIMIT 20;
CREATE ROLE myapp_dba WITH LOGIN PASSWORD 'DBAPass123!' CREATEDB CREATEROLE;

-- 4. Assign to groups
GRANT myapp_write TO myapp_api;
GRANT myapp_write TO myapp_worker;
GRANT myapp_admin TO myapp_dba;

-- 5. Database access
GRANT CONNECT ON DATABASE myapp TO myapp_read;

-- 6. Schema access
GRANT USAGE ON SCHEMA public TO myapp_read;

-- 7. Existing tables
GRANT SELECT ON ALL TABLES IN SCHEMA public TO myapp_read;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO myapp_write;
GRANT ALL ON ALL TABLES IN SCHEMA public TO myapp_admin;

-- 8. Existing sequences
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO myapp_write;

-- 9. Default privileges (for future objects)
ALTER DEFAULT PRIVILEGES FOR ROLE myapp_dba IN SCHEMA public
GRANT SELECT ON TABLES TO myapp_read;
ALTER DEFAULT PRIVILEGES FOR ROLE myapp_dba IN SCHEMA public
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO myapp_write;
ALTER DEFAULT PRIVILEGES FOR ROLE myapp_dba IN SCHEMA public
GRANT ALL ON TABLES TO myapp_admin;
ALTER DEFAULT PRIVILEGES FOR ROLE myapp_dba IN SCHEMA public
GRANT USAGE, SELECT ON SEQUENCES TO myapp_write;
```

Superuser and Special Roles

Superuser

```
-- Superuser bypasses ALL security checks
-- NEVER use for application connections
CREATE ROLE superadmin WITH SUPERUSER LOGIN PASSWORD 'SuperPass!';

-- Check if current user is superuser
SELECT current_setting('is_superuser');
SELECT usesuper FROM pg_user WHERE username = current_user;
```

Built-in Special Roles (PostgreSQL 14+)

```
-- pg_read_all_data: SELECT on all tables in all databases
GRANT pg_read_all_data TO readonly_admin;

-- pg_write_all_data: INSERT/UPDATE/DELETE on all tables
GRANT pg_write_all_data TO etl_user;

-- pg_read_all_settings: Read all configuration parameters
GRANT pg_read_all_settings TO monitoring_user;

-- pg_read_all_stats: Read all pg_stat_* views
GRANT pg_read_all_stats TO monitoring_user;

-- pg_stat_scan_tables: Run pg_stat_user_tables
GRANT pg_stat_scan_tables TO monitoring_user;

-- pg_monitor: All monitoring privileges (combines above)
GRANT pg_monitor TO monitoring_user;

-- pg_signal_backend: Send signals to backend processes
GRANT pg_signal_backend TO dba_user; -- Can pg_cancel_backend, pg_terminate_backend

-- View all predefined roles
SELECT rolname, rolsuper FROM pg_roles WHERE rolname LIKE 'pg_%';
```

Privilege Inspection Queries

```
-- Who has what privileges on a table
SELECT grantee, privilege_type, is_grantable
FROM information_schema.role_table_grants
WHERE table_name = 'orders' AND table_schema = 'public';

-- Expanded view of table ACLs
SELECT
```

```

        n.nspname AS schema,
        c.relname AS table,
        c.relacl AS access_control_list
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
      AND n.nspname = 'public'
ORDER BY c.relname;

-- Role membership tree
WITH RECURSIVE role_tree AS (
    SELECT r.rolname, r.rolname AS parent, 0 AS depth
    FROM pg_roles r
    WHERE r.rolname = 'myapp_admin'
    UNION ALL
    SELECT m.rolname, rt.rolname, rt.depth + 1
    FROM pg_auth_members am
    JOIN pg_roles m ON m.oid = am.member
    JOIN role_tree rt ON rt.rolname = (SELECT rolname FROM pg_roles WHERE oid =
am.roleid)
)
SELECT depth, rolname FROM role_tree ORDER BY depth, rolname;

-- All permissions for a specific user (including inherited)
SELECT
    table_schema,
    table_name,
    string_agg(privilege_type, ', ' ORDER BY privilege_type) AS privileges
FROM information_schema.role_table_grants
WHERE grantee = 'appuser'
      OR grantee IN (
        SELECT r.rolname
        FROM pg_auth_members am
        JOIN pg_roles r ON r.oid = am.roleid
        JOIN pg_roles m ON m.oid = am.member
        WHERE m.rolname = 'appuser'
    )
GROUP BY table_schema, table_name
ORDER BY table_schema, table_name;

-- Database-level privileges
SELECT datname,
       pg_catalog.pg_get_userbyid(datdba) AS owner,
       pg_catalog.array_to_string(datacl, E'\n') AS access_privileges
FROM pg_catalog.pg_database
ORDER BY datname;

-- Check if user can access schema
SELECT has_schema_privilege('appuser', 'public', 'USAGE');

-- Check table privilege
SELECT has_table_privilege('appuser', 'orders', 'SELECT');

```

```
SELECT has_table_privilege('appuser', 'orders', 'INSERT');

-- Check column privilege
SELECT has_column_privilege('appuser', 'users', 'password_hash', 'SELECT');
-- Returns false → user correctly blocked from password column
```

Common Mistakes

1. **Granting to PUBLIC** — PUBLIC is implicit group all users belong to; grants to PUBLIC affect everyone
 2. **Using superuser for application** — if app is compromised, entire server is compromised
 3. **Not revoking PUBLIC on schema** — `REVOKE CREATE ON SCHEMA public FROM PUBLIC`
 4. **Not using group roles** — assigning privileges directly to users makes management hard
 5. **Forgetting default privileges** — new tables created by app owner aren't accessible to app user
 6. **Granting all on pg_catalog** — allows reading password hashes
 7. **GRANT OPTION cascade revoke** — revoking with CASCADE removes all transitively granted privileges
 8. **Not testing privilege requirements** — use `SET ROLE` to test as a lower-privileged role
-

Best Practices

1. **Principle of least privilege** — grant only what's necessary
 2. **Use group roles** — grant to groups, assign users to groups
 3. **Never use superuser for apps** — create dedicated application roles
 4. **Revoke public schema create** — prevent users from creating objects in public schema
 5. **Use pg_read_all_data** for read-only admin roles (PostgreSQL 14+)
 6. **Set up default privileges** for consistent access to new tables
 7. **Regularly audit privileges** — run privilege inspection queries quarterly
 8. **Use SECURITY DEFINER carefully** — it runs as function owner, can be exploited
 9. **Set search_path in SECURITY DEFINER functions** — prevent search_path injection
 10. **Document role hierarchy** — maintain a privilege matrix
-

Interview Questions

Q1: What is the difference between a user and a role in PostgreSQL?

A: In PostgreSQL, users and roles are the same thing — both are "roles" in `pg_authid`. The distinction is: login roles (users) have `rolcanlogin=true` and can authenticate and connect. Group roles have `rolcanlogin=false` and are used to hold and distribute privileges. `CREATE USER` is just shorthand for `CREATE ROLE ... LOGIN`.

Q2: What is role inheritance and when would you disable it?

A: With `INHERIT` (default), a role automatically gets all privileges of roles it's a member of. With `NOINHERIT`, the role must explicitly `SET ROLE` to activate parent privileges. Disable inheritance for security-sensitive roles where you want explicit privilege escalation (audit trail) — e.g., a DBA who must consciously switch to a high-privilege role rather than always having it active.

Q3: What are default privileges and why are they important?

A: `ALTER DEFAULT PRIVILEGES` sets automatic privilege grants for future objects created by a specific role in a specific schema. Without them, each new table created by the app owner would need manual grants. With

them, newly created tables automatically get the correct permissions. This is critical for ORM-managed schemas where new tables are created regularly.

Q4: Explain the `WITH GRANT OPTION` .

A: `GRANT SELECT ON TABLE t TO user1 WITH GRANT OPTION` allows `user1` to in turn grant `SELECT` on that table to other users. Without `WITH GRANT OPTION` , a user can use the privilege but not delegate it. Important: if you revoke from `user1 CASCADE` , it also removes grants `user1` made to others.

Q5: What is `SECURITY DEFINER` and what are its risks?

A: A function with `SECURITY DEFINER` executes with the privileges of the function's owner, not the caller. This allows unprivileged users to call a function that performs privileged operations (like accessing a table they can't directly query). Risk: if the function has a SQL injection vulnerability, the attacker executes as the owner. Mitigation: always set `SET search_path = pg_catalog, public` in `SECURITY DEFINER` functions to prevent schema search manipulation.

Q6: How would you implement read-only access to all tables without superuser?

A: PostgreSQL 14+: `GRANT pg_read_all_data TO readonly_user;` . Older: (1) Create a `readonly` group role. (2) `GRANT USAGE ON SCHEMA public TO readonly;` . (3) `GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;` . (4) Set up default privileges so future tables are also accessible.

Q7: What happens when you `REVOKE privileges FROM PUBLIC` ?

A: `PUBLIC` is an implicit group that all users belong to. Any grants made to `PUBLIC` apply to all current and future users. Revoking from `PUBLIC` removes the privilege for all users (they would need explicit grants). Important initial hardening: `REVOKE CREATE ON SCHEMA public FROM PUBLIC;` prevents users from creating objects in the public schema.

Q8: How do you audit what privileges a user has in PostgreSQL?

A: Query `information_schema.role_table_grants` for table privileges. Use `has_table_privilege()` function for quick checks. For roles, query `pg_auth_members` to see group memberships. Use `\dp tablename` in psql for a privilege summary. Write a recursive CTE to trace the full role hierarchy and cumulative privileges.

Exercises and Solutions

Exercise 1: Implement RBAC for a Multi-Team Application

```
-- Teams: backend, frontend, data_analyst, devops

-- Groups
CREATE ROLE backend_dev;
CREATE ROLE frontend_dev;
CREATE ROLE data_analyst;
CREATE ROLE devops;

-- Privileges per group
-- Backend: full app tables, no admin tables
GRANT USAGE ON SCHEMA public TO backend_dev;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE orders, products, customers TO
```

```

backend_dev;

-- Frontend: read-only on safe tables
GRANT USAGE ON SCHEMA public TO frontend_dev;
GRANT SELECT ON TABLE products, categories TO frontend_dev;

-- Data analyst: read-only on all tables + analytics schema
GRANT USAGE ON SCHEMA public, analytics TO data_analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO data_analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA analytics TO data_analyst;

-- DevOps: can create schemas but not read data
GRANT CREATE ON DATABASE mydb TO devops;

-- Users
CREATE ROLE alice WITH LOGIN PASSWORD 'AlicePass!' IN ROLE backend_dev;
CREATE ROLE bob WITH LOGIN PASSWORD 'BobPass!' IN ROLE frontend_dev;
CREATE ROLE carol WITH LOGIN PASSWORD 'CarolPass!' IN ROLE data_analyst;

```

Exercise 2: Privilege Audit Query

```

-- Show all roles, their groups, and table privileges
SELECT DISTINCT
    u.rolname AS username,
    g.rolname AS group,
    t.table_schema,
    t.table_name,
    t.privilege_type
FROM pg_roles u
JOIN pg_auth_members am ON am.member = u.oid
JOIN pg_roles g ON g.oid = am.roleid
JOIN information_schema.role_table_grants t ON t.grantee = g.rolname
WHERE u.rolcanlogin = true
    AND t.table_schema NOT IN ('information_schema', 'pg_catalog')
ORDER BY u.rolname, g.rolname, t.table_name;

```

Cross-References

- [01 authentication methods.md](#) — Authentication before authorization
- [03 row level security.md](#) — Row-level privilege enforcement
- [06 auditing.md](#) — Auditing privilege usage
- [08 compliance considerations.md](#) — Compliance requirements for access control